



Security Review Report

Panther Protocol Infrastructure Security Review

Prepared by:

Arcadia

Prepared for:

Panther Protocol Foundation

Date: April 22nd, 2026

<https://arcadia.agency/>

Email: audits@arcdiamgroup.com

Telegram: <https://t.me/thearcadiagroup>



Auditor Information

Company: Arcadia

Point of Contact: Rasikh Morani, Co-Founder

Address: 955 W John Carpenter Frwy, Ste. 100, Irving, Texas 75039

Website: <https://arcadia.agency/>

Email: audits@arcdiamgroup.com

About Us: Arcadia is a software security and product development company building and securing software.

Client Information

Company Name: Panther Protocol Foundation

Website: <https://www.pantherprotocol.io/>

Project Name: Panther Protocol dApp



Revision History

Date	Reason	Commit
04/07/2026	Initial Audit Scope and Engagement	
04/07/2026	Remediation of PP-01 (build pipeline credential exposure)	#5ac1c12
04/07/2026	Partial Remediation of PP-11 (bundler proxy migration)	#a83e335
04/22/2026	Final Audit Report Delivered	

Contents

Executive Summary	6
1 Introduction	8
1.1 Audit Scope	8
1.2 Out of Scope	9
1.3 Project Background	10
1.4 Coverage	10
2 Methodology	12
2.1 Investigation structure	12
2.2 Track A: codebase content analysis	12
2.2.1 Repository acquisition and integrity verification	12
2.2.2 Commit signing baseline	13
2.2.3 Identity-based reconnaissance	13
2.2.4 Off-hours commit pattern analysis	13
2.2.5 Sensitive-file modification audit	13
2.2.6 Per-commit content review: the four-gate test	14
2.2.7 Verification protocol for suspicious findings	14
2.2.8 Threat-surface audits	15
2.3 Track B: GitLab server-side audit log analysis	16
2.3.1 Access and tooling	16
2.3.2 Chain of custody	16
2.3.3 Group and project enumeration	16
2.3.4 Audit event collection	17
2.3.5 Push event collection	17
2.3.6 Pipeline collection	17
2.3.7 IP geolocation and team baseline	17
2.3.8 Out-of-band Personal Access Token usage verification	18
2.4 Synthesis and reconciliation	18
2.5 Acknowledged limitations	18
2.6 Key Security Concerns	19
3 Report Summary	21
4 Audit Findings	21
4.1 [PP-01] Build pipeline serialized all CI/CD environment variables into deployed bundles	21
4.2 [PP-02] Default branch is not a protected branch	23
4.3 [PP-03] No commit signing required or in use	24
4.4 [PP-04] CI/CD variables marked unprotected	25
4.5 [PP-05] No CODEOWNERS rules enforce review on security-sensitive files	26
4.6 [PP-06] Sentry initialization lacks beforeSend hook	28



4.7	[PP-07] Two-factor authentication not required for group members	29
4.8	[PP-08] Project Access Token with overly broad scope	30
4.9	[PP-09] Development-mode logging serializes wallet keypair structures	31
4.10	[PP-10] Sibling repository CI/CD variables marked unprotected	32
4.11	[PP-11] Bundler API endpoint hardcoded with leftover debug logging	33
5	Forensic Verification	35
5.1	Verification approach	35
5.2	Hypotheses tested	35
5.3	Evidence collected	36
5.4	Findings per hypothesis	37
5.5	Acknowledged limitations of the verification	42
5.6	Conclusion of the verification	43
6	Conclusion	44
7	Contact Information	45
8	Disclaimer	45

Executive Summary

The Panther Protocol Foundation engaged Arcadia to perform an infrastructure security review of the Panther Protocol dApp and supporting GitLab repositories following an external disclosure of a credential exposure in deployed build artifacts. A misconfiguration in the dApp’s webpack build pipeline had caused all CI/CD environment variables present at build time (including a GitLab Personal Access Token with Maintainer-level access to all seven Foundation repositories, third-party storage credentials, error-monitoring tokens, and RPC provider keys) to be embedded in every IPFS-deployed JavaScript bundle for an approximately 16-month window beginning November 2024. Because IPFS-deployed bundles are immutable, the leaked credentials remained accessible from any historical artifact even after the underlying configuration was corrected. The bug was identified externally, disclosed to the Foundation, and the immediate root cause was remediated by the Foundation’s engineering team within hours of disclosure. Arcadia’s engagement covered (a) verification of the remediation, (b) a comprehensive forensic review to determine whether the exposed credentials had been used to inject malicious code into the dApp source repository during the exposure window, and (c) a broader security review of the Foundation’s GitLab infrastructure and dApp codebase to identify additional latent issues.

Across seven independent forensic surfaces (a per-commit content review of every commit on the default branch during the exposure window, a build pipeline audit, a network and exfiltration sink audit, a private-key handling and inter-process communication audit, a push event timing and continuity analysis, a CI/CD pipeline and secret extraction analysis, and a server-side audit log review across all seven Foundation repositories) Arcadia found no evidence of malicious code injection during the exposure window. The analysis was performed using automated tooling across the full corpus of commits, push events, pipeline runs, and audit events; every commit, event, or artifact flagged as anomalous by the automated passes was then independently verified by Arcadia personnel against the underlying diff, the surrounding code, and any supporting context before being recorded as a finding or dismissed as a false positive. Direct evidence was identified that the exposed credentials had been used to authenticate to the GitLab API from non-team source addresses; the source addresses correspond to a party already known to the Foundation. No state-changing operations (pushes, merge request merges, CI variable modifications, member changes, settings changes) were observed under the affected identity from the anomalous source addresses, across any of the seven Foundation repositories, during the visible portion of the exposure window. The available evidence is consistent with read-only use of the exposed credentials. Arcadia identified eleven additional findings ranging from informational to high severity, primarily relating to repository hardening, branch protection, commit signing, CI/CD variable scoping, monitoring configuration, and code review practices. As part of the remediation effort, the Foundation migrated



its repositories from GitLab to GitHub; several of the infrastructure-hardening findings were addressed in the course of that migration. Five findings are fully remediated (including the original credential exposure), two are partially remediated, three are acknowledged as accepted risks, and one is planned for a later hardening phase. This report includes the detailed methodology, scope, findings, recommendations, and forensic verification record.



1 Introduction

Following an external disclosure of a credential exposure in deployed build artifacts, the Panther Protocol Foundation engaged Arcadia to assess the remediation and to verify whether the exposed credentials had been used to compromise the codebase during the exposure window. The engagement extended beyond the immediate verification question to include a broader security review of the dApp source repository, the supporting GitLab infrastructure, and the seven repositories under the Panther Protocol Foundation organization on GitLab.

The work was structured as a two-track investigation. The first track focused on the codebase content: cloning the dApp repository, performing identity-based reconnaissance of its commit history, content-reviewing every commit on the default branch within the exposure window, and auditing the live source code for malicious modifications, exfiltration paths, and inter-process communication weaknesses. The second track focused on the GitLab infrastructure itself: enumerating projects, members, audit events, push events, pipeline runs, CI/CD variables, project and group access tokens, and the geolocation of every source address that authenticated to any Foundation resource during the visible portion of the exposure window. The two tracks were deliberately structured to be independent of each other so that the findings of one would not bias the findings of the other; the conclusions were reconciled at the synthesis stage.

1.1 Audit Scope

The audit scope covered the Panther Protocol dApp source repository in its current state and across the full exposure window of the credential exposure incident, the configuration of the supporting GitLab infrastructure, and the operational state of the six sibling repositories under the same Foundation organization. In particular, the scope included:

- **The Panther Protocol dApp repository:** full git history (3,988 commits across all branches), all source code at HEAD, the build configuration (webpack, package.json, yarn.lock, .gitlab-ci.yml), the deployment scripts, and all branch and tag references. Particular attention was paid to commits authored on or merged into the default branch during the credential exposure window of November 2024 through March 2026.
- **Six sibling Foundation repositories:** audit events, member lists, push event histories, CI/CD variable enumeration, project access tokens, branch protection configuration, and protected branch history.
- **GitLab group-level configuration:** group settings, member roster, audit events, group access tokens, branch protection defaults, and required two-factor enforcement state.



- **Build and deployment pipeline:** webpack configuration, package.json scripts, .gitlab-ci.yml, IPFS deployment scripts, install lifecycle hooks (preinstall, postinstall, prepare), and yarn.lock dependency resolution sources.
- **Wallet engine and cryptographic surfaces:** private key derivation and handling, ZK proof witness generation, BroadcastChannel listeners, postMessage flows between main thread and Web Workers, persistent storage of sensitive material, and any code path that could exfiltrate cryptographic state outside its intended worker context.
- **Network and external service surfaces:** every fetch, XMLHttpRequest, WebSocket, sendBeacon, dynamic import(), and dynamic script tag injection in the dApp source, with destination allowlist verification and data flow analysis.
- **Error reporting and telemetry:** the Sentry SDK initialization, the configured DSN endpoints, the integration set, and the beforeSend hook configuration.

Source Code Repositories

- Panther Protocol dApp (private repository hosted on the Foundation's GitLab organization)
- Six sibling repositories under the same organization (audit-only, content review out of scope for this engagement)

1.2 Out of Scope

The following were explicitly out of scope for this engagement and may warrant separate review in a future engagement:

- The Panther Protocol on-chain smart contracts and their formal verification
- The trusted setup ceremony participants, ceremony output verification, and proving key integrity
- Third-party services whose credentials were exposed (Pinata, Firebase, the configured Sentry organization, Etherspot bundler, RPC providers); access logs and forensic review of those services lie outside Arcadia's scope and would require coordination with the respective service providers
- The dApp's UI/UX and accessibility
- Any non-Foundation GitLab namespace that may have hosted the dApp project prior to the May 2025 organization migration



1.3 Project Background

Panther Protocol is a privacy-focused protocol implementing zero-knowledge proof-based shielded transactions on Ethereum-compatible chains. The Panther Protocol dApp is the official user-facing web application that interacts with the protocol's smart contracts, manages users' shielded accounts, generates zero-knowledge proofs for transactions, and orchestrates the wallet engine that maintains local state for shielded balances and unspent transaction outputs. The dApp is deployed as a static JavaScript bundle pinned to IPFS, with users accessing it via configured IPFS gateways. At the time of this engagement the dApp was in a pre-launch canary phase with limited public exposure, but historical bundles from earlier phases of the project remain accessible on IPFS and contain the credentials exposed by the underlying issue this audit was triggered by.

1.4 Coverage

This audit comprehensively reviewed the dApp source repository, build pipeline, supporting GitLab infrastructure, and a forensic verification of code integrity across the credential exposure window. The review included the following work in detail:

- **Local repository clone analysis:** the entire dApp git repository was cloned and analyzed: commit signing status (across all 3,988 historical commits), author identity inventory, off-hours commit analysis cross-referenced against authors' expected timezones, sensitive-file modification history, force-push detection via the local reflog and via cross-reference against server-side push event records, and `git fsck` object integrity verification.
- **Per-commit content review:** every commit on the default branch within the exposure window was individually content-reviewed against a four-gate test: (1) does the diff perform what its message claims, (2) is the diff scoped to the message's stated purpose, (3) does it modify any security-sensitive surface without explicit justification, and (4) does it introduce any suspicious primitives such as `eval`, dynamic `import()` with non-literal arguments, new `BroadcastChannel` listeners, or external network targets not on the existing allowlist.
- **Build and deploy pipeline audit:** `webpack.config.ts`, `.gitlab-ci.yml`, IPFS deployment scripts, `package.json`, `yarn.lock`, and all install life-cycle hooks were audited for any pattern that could fetch or execute remote code at install or build time, exfiltrate secrets via job logs, or modify the deployed bundle in ways inconsistent with the project's stated purpose.
- **Network and exfiltration surface audit:** every network IO call site in the dApp's source tree was inventoried, classified by destination, and



verified against the legitimate destination allowlist. Suspicious primitives (`eval`, `new Function`, `dynamic import`, `sendBeacon`, dynamic script injection, base64-decoded URLs, references to webhook services or paste-bins) were grepped for and reviewed.

- **Wallet engine and cryptographic surface audit:** a full data flow trace of `PrivateKey` types and `Keypair` objects from generation through every read site and message-passing boundary, an inventory of every `BroadcastChannel` and worker `postMessage` flow, verification that ZK proof witnesses never exit the worker context, and an audit of persistent storage encryption.
- **GitLab server-side audit log analysis:** 226 audit events across the group and all seven projects were collected with full chain-of-custody (raw API responses preserved with SHA-256 hashes), filtered by event type, cross-referenced by author identity and source IP address, and analyzed for anomalies in actor-IP-action triples.
- **Push event analysis:** 523 push events for the dApp project were collected from the GitLab events feed and analyzed for author identity consistency, branch ref distribution, commit hash continuity against the local git history (to detect force-push artifacts and orphan commits), suspect-account activity, and timezone consistency per author.
- **CI/CD pipeline forensics:** 511 pipeline runs were collected and analyzed by triggering user, source type (push, web, API, schedule, trigger, MR event), branch ref, and job content. The `.gitlab-ci.yml` history was cross-checked for any modifications that could enable secret exfiltration via job logs.
- **Sibling project audits:** the six other Foundation repositories were each audited for audit events, push events, member lists, CI/CD variables, and any sign of activity by suspect identities or source addresses.
- **IP geolocation and team baseline construction:** every unique source IP that appeared in any audit event was geolocated via two independent geolocation providers and cross-referenced against expected team locations to build a baseline of legitimate vs anomalous IPs. Particular attention was paid to any IP appearing under more than one team identity (the strongest signature of credential theft via the lazy version of the attack).
- **Personal Access Token enumeration:** Project Access Tokens and Group Access Tokens were enumerated across all seven projects and the parent group. Personal Access Tokens were not enumerable via the API (a documented limitation of the GitLab tier in use, discussed in the Methodology section).
- **Out-of-band PAT usage record verification:** the IP usage history



of the specific exposed Personal Access Token was obtained out-of-band from the token owner via the GitLab user interface and preserved with chain of custody. Each IP in the usage record was independently geolocated and classified.

2 Methodology

This section describes Arcadia’s investigation methodology in detail. It is included at the depth necessary for an external reader to understand the basis on which the report’s conclusions are drawn, and to enable independent verification or replication of any individual finding.

2.1 Investigation structure

The investigation followed a deliberate two-track structure designed to surface the same conclusions through independent methods, so that any single finding could be cross-validated against another track:

- **Track A: Codebase content analysis.** Cloning the dApp repository and examining its source code, commit history, build configuration, and runtime behavior directly. This track answers “what does the code look like, and what does the commit history show.”
- **Track B: GitLab server-side audit log analysis.** Querying the GitLab REST API for audit events, push events, pipeline runs, member changes, token usage, and IP-level activity records. This track answers “what actually happened on the server, by whom, and from where.”

The two tracks have different failure modes. Track A can detect injected code that exists in the repository but cannot tell who authored it or from what IP address. Track B can detect anomalous push or authentication activity but cannot determine whether the resulting code is malicious. Both tracks were necessary for a complete answer; either track alone would have left a verification gap.

2.2 Track A: codebase content analysis

2.2.1 Repository acquisition and integrity verification

The dApp repository was cloned in full, including all branches and tags. `git fsck -full -strict` was run against the local clone to verify that the repository’s object store was internally consistent (no broken parent chains, no corrupted tree objects, no missing referenced objects). The result was clean: the local clone passed integrity verification with no errors, providing a baseline indicating that no historical commit had been tampered with at the git object level since the clone.



2.2.2 Commit signing baseline

Across all 3,988 commits in the repository's history (every author, every branch, every tag, every era), signing status was checked using `git log --pretty=format:'%G?'`. The result: zero commits carry GPG or SSH signatures. The signing baseline is 0%, meaning that signing-based detection of impersonation is unavailable for this repository. A stolen credential pushing commits under any identity would produce commits indistinguishable from legitimate ones in their metadata. This is documented as an architectural finding (PP-03) and significantly shaped the investigative methodology: the analysis had to rely on content review and IP-layer analysis rather than on signature verification.

2.2.3 Identity-based reconnaissance

The set of unique committer email addresses across the entire repository history was enumerated, the per-author commit count tabulated, and the first-seen date for each contributor identified. The set of authors active during the exposure window was compared against the set active before it. No new author identities appeared during the exposure window beyond a small number of contributors who could be cross-referenced against known team members. This rules out the simplest impersonation attempt (an attacker pushing under a brand-new identity) but does not rule out impersonation under an existing identity, which the investigation addressed via separate methods.

2.2.4 Off-hours commit pattern analysis

For each contributor active during the exposure window, the distribution of commit timestamps was tabulated against UTC time-of-day. Each contributor's normal working-hours band was inferred from their cumulative pattern, and any individual commits outside the normal band were flagged for review. This pattern analysis identified one contributor whose UTC timestamps appeared off-hours but which corresponded perfectly to normal working hours in their local timezone (-03:00, South America). No off-hours anomalies were found for any other contributor, providing partial assurance that nobody was pushing under an impersonated identity at the wrong time of day for that identity.

2.2.5 Sensitive-file modification audit

Every commit during the exposure window that touched any of the following file paths was identified and individually examined: `webpack.config.ts`, `.gitlab-ci.yml`, `package.json`, `yarn.lock`, the wallet engine source files, the BroadcastChannel and inter-worker communication files, the private-key handling source files, and the ZK proof generation pipeline. For each commit, the diff was reviewed, the commit message verified against the diff, and the author cross-referenced against the file's typical maintainer set.



2.2.6 *Per-commit content review: the four-gate test*

Every commit on the default branch within the exposure window (270 commits in total) was subjected to a structured content review. Each commit was passed through a four-gate test:

- **Gate 1 (Diff matches stated purpose):** does the diff actually do what the commit message claims it does? (A “fix typo” commit should not add a feature.)
- **Gate 2 (Scope discipline):** does the diff do *only* what the message says, or does it also touch unrelated files or code? (Scope creep on a security-sensitive surface is a flag.)
- **Gate 3 (Sensitive-surface justification):** does the diff modify any of the predefined sensitive surfaces (build configuration, key handling, IPC, ZK proof pipeline, telemetry, network connectors, wallet provider methods) without an explicit justification in the commit message?
- **Gate 4 (Suspicious primitive introduction):** does the diff add any of the following: `eval()`, `new Function()`, `dynamic import()` with a non-literal argument, `new WebSocket()`, `navigator.sendBeacon()`, `fetch()` to a domain not in the existing allowlist, dynamic script tag creation with `.src` assignment, base64-encoded URLs that get decoded and used as request targets, new `BroadcastChannel` instantiations, new global `message` listeners, hex strings or base64 strings of typical key length in non-test code, or references to webhook services, pastebins, or commercial proxy/tunnel domains?

Each commit was classified as Green (passes all gates), Yellow (passes the gates but touches a sensitive surface; flagged for human verification), Orange (fails Gate 2 or has unexplained sensitive-surface modifications), or Red (fails Gate 1 or introduces unjustified suspicious primitives). All Yellow, Orange, and Red classifications were independently verified by Arcadia personnel against the actual diff, the surrounding code in the current HEAD, and any cross-references in the disclosure materials.

2.2.7 *Verification protocol for suspicious findings*

During the per-commit review and during the targeted threat-surface audits, several findings were initially escalated to “confirmed injection” classifications based on pattern matching. Each such escalation was independently verified by Arcadia personnel before being treated as a finding. The verification protocol for any escalated finding was: (1) verify the change against the rest of the codebase to determine whether the same pattern exists elsewhere, (2) verify against the project’s stated conventions and ecosystem norms (Web3 dApps have specific norms around hardcoded RPC keys, public IPFS gateways, and wallet provider injection that differ from generic web application security expectations), (3) verify against the file’s modification history to determine whether



the change is a continuation of an established pattern or a deviation from it, and (4) verify against any supporting documentation or disclosure context. Several initial escalations were determined to be false positives under this protocol: the original credential exposure misconfiguration (which fits the textbook profile of an honest mistake during a routine package upgrade); a legitimate Sentry organization rename and US-to-EU region migration consistent with GDPR data residency requirements; and a hardcoded RPC provider configuration that turned out to use a key already present elsewhere in the codebase and that follows standard Web3 dApp practice for client-side rate-limit identifiers. These verification outcomes are reflected in the findings list rather than escalated as security incidents.

2.2.8 Threat-surface audits

In addition to the per-commit content review, five parallel content audits were performed on dedicated threat surfaces:

- **Build and deploy pipeline:** `webpack.config.ts` read in full; `.gitlab-ci.yml` reviewed for any stage that fetches external scripts, runs `curl|bash` patterns, uses suspicious docker images, communicates with non-Panther domains, or runs at CI time without a clear legitimate purpose; deployment scripts (notably `dapp/scripts/deployIpfs.ts`) reviewed for any path that ships the bundle anywhere besides legitimate IPFS pinning; `package.json` reviewed for typosquatted dependencies, unusual maintainers, low-version-number packages added during the window, and any `preinstall`, `postinstall`, or `prepare` lifecycle script that could execute arbitrary code at install time; `yarn.lock` reviewed for any resolved URL not pointing to the official npm/yarn registry.
- **Network and exfiltration sinks:** every `fetch`, `XMLHttpRequest`, `WebSocket`, `navigator.sendBeacon`, `dynamic import()`, `new Function`, `eval`, `dynamic document.createElement('script')`, `new Image` pixel pattern, and `postMessage` call site in `dapp/src/` was inventoried; for each, the destination URL or computed target was traced, the data being sent was inspected, and the destination was verified against an allowlist of legitimate Panther and third-party endpoints.
- **Private key handling and inter-process communication:** every reference to the `PrivateKey` type, the `Keypair` type, mnemonic seed material, and key derivation routines was traced from generation through every read site; every `BroadcastChannel` instantiation was inventoried with its name, its writers, its listeners, and the data shape flowing over it; every worker `postMessage` flow was traced with its sender, its receiver, and the data shape; every `localStorage`, `sessionStorage`, and `IndexedDB` read and write was reviewed for whether sensitive material is stored encrypted or in plaintext.
- **ZK proof witness handling:** the proof generation pipeline in `dapp/src/services/zkProo`

and the ZK worker code were reviewed to verify that proof witnesses (which contain private inputs) never leave the worker context generating them, and that only public proof outputs are exported.

- **Per-commit content review:** described above.

2.3 Track B: GitLab server-side audit log analysis

2.3.1 Access and tooling

A GitLab Personal Access Token with the necessary scopes was obtained from a Foundation member with Owner-level access to the parent group. The token was stored in a file with restrictive permissions (mode 0600) for the duration of the engagement and was queued for revocation immediately upon completion of the work. All API queries were made via `curl` with the `PRIVATE-TOKEN` header. The queries used were saved verbatim to a query log file with UTC timestamps for reproducibility.

2.3.2 Chain of custody

A forensic evidence directory was established with the following structure: a raw evidence subdirectory (`00-raw/`) containing immutable copies of every API response received from GitLab, a processed subdirectory (`01-processed/`) for derived data, an analysis subdirectory (`02-analysis/`) for per-hypothesis analysis files, and a queries subdirectory (`03-queries/`) for saved query commands. Each raw file was immediately hashed with SHA-256 upon collection, with the hash recorded in a manifest file (`EVIDENCE-MANIFEST.md`). Every command run against the GitLab API was logged in an append-only query log (`QUERY-LOG.md`) with a UTC timestamp, the exact command, the source endpoint, and the resulting file's hash. This discipline allows any individual finding to be traced back to a specific raw API response and an explicit, replayable query.

2.3.3 Group and project enumeration

The Foundation's GitLab group was enumerated for its current configuration: visibility, default branch protection settings, two-factor enforcement, and the full project list. Each of the seven projects was enumerated for its creation date, current member roster, current CI/CD variable list, current project access tokens, and current protected branch configuration. An important constraint surfaced during this enumeration: the current group was created in April 2025, and the dApp project was created in this group in May 2025; both dates fall *after* the start of the credential exposure window. This means that audit data prior to May 2025 (approximately the first six months of the exposure window) is not in the current group's audit log because the project did not exist in this namespace yet. This constraint is acknowledged in the limitations section.

2.3.4 Audit event collection

Audit events were paginated and bulk-fetched for the entire exposure window from both the group-level and the project-level endpoints. Forty group-level events and 186 project-level events (across all seven projects) were collected, hashed, and saved to raw evidence files. The events were inventoried by event type, by author identity, by source IP address, and by date. Particular attention was paid to event types capturing membership changes, settings changes, CI variable creation/update/deletion, project access token creation, protected branch modification, and project fork operations.

2.3.5 Push event collection

The GitLab Premium tier in use does not capture individual git push events at the audit-event level (the `repository_git_operation` event class is restricted to the Ultimate tier). Push events were instead obtained from the project events feed (`/projects/{id}/events?action=pushed`), which provides authoring identity, timestamp, branch ref, commit count, and commit hash range, but does *not* include source IP addresses. 523 push events for the dApp project were collected across the exposure window, plus 227 push events for the six sibling projects (151 for one repository, 45 for another, and progressively fewer for the others).

2.3.6 Pipeline collection

511 pipeline runs for the dApp project across the exposure window were collected via the pipelines API and individually inspected for triggering user, triggering source (push, web, API, schedule, trigger, merge_request_event), branch ref, and status. Pipelines on suspect branches or from suspect users were further drilled into via the jobs API to inspect job content for any pattern that could exfiltrate secrets via job logs (printenv, env, base64-encoded data, curl with secrets in URLs, etc.).

2.3.7 IP geolocation and team baseline

Every unique source IP address that appeared in any audit event was extracted, deduplicated, and geolocated via two independent providers (`ipinfo.io` as primary and `ip-api.com` as fallback/cross-check). For each IP, the country, city, ISP, ASN, and IP type (residential, datacenter, mobile, VPN exit) were captured. A team baseline was constructed mapping each known team member to their expected geographic pattern, and each IP was classified as one of: `TEAM_CONFIRMED` (matches expected pattern of a known member), `TEAM_PLAUSIBLE` (consistent with travel or known VPN usage by a known member), `DATACENTER_INFRASTRUCTURE` (resolves to a cloud provider's known IP range belonging to GitLab's own infrastructure), or `ANOMALOUS` (does not match any known pattern). The single most diagnostic check

was whether any IP appeared under more than one team identity, the strongest signature of credential theft via the lazy version of the attack.

2.3.8 Out-of-band Personal Access Token usage verification

The GitLab REST API does not permit an Owner-level token holder to enumerate other users' Personal Access Tokens; this capability is reserved to GitLab instance administrators or to the token-owning user themselves. To work around this limitation, the IP usage record for the specific exposed Personal Access Token was obtained out-of-band: the token's owner accessed their own GitLab user settings page, captured the IP usage panel for the token, and forwarded the data via a Foundation member with chain-of-custody discipline. The captured data was preserved in the forensic evidence directory and each IP was independently geolocated and classified.

2.4 Synthesis and reconciliation

The findings of Tracks A and B were reconciled at the synthesis stage. The two tracks tell complementary halves of the same story: the audit-event data tells us about state-changing operations and which IPs performed them, while the codebase content review tells us whether the resulting state changes contain malicious content. The PAT usage record (out-of-band) tells us about authentication events, including read-only authentications that do not generate audit events. Together, the three data sources allow conclusions about both the state of the codebase and the historical use of the exposed credentials, within the explicit limitations described in the next section.

2.5 Acknowledged limitations

The methodology has several explicit limitations that the conclusions must be interpreted against:

- **Read-only API operations do not generate audit events.** GitLab Premium tier captures state-changing operations (pushes, MR merges, CI variable changes, member changes, settings changes) but does not generate audit events for read operations such as repository content reads via the API, CI variable value reads, project metadata browsing, or job log retrievals. As a result, the investigation cannot directly enumerate which API endpoints were accessed via the exposed credentials.
- **The first six months of the exposure window are not in the current group's audit log.** The Foundation's current GitLab group was created in April 2025; the dApp project was migrated into the group in May 2025. Audit data from the start of the exposure window (November 2024) through May 2025 lives in whichever GitLab namespace the project was previously hosted in, which is outside the scope of this engagement. Recovering this data would require either identifying the pre-



decessor namespace and pulling its audit log, or requesting server-side records from GitLab security support.

- **Personal Access Token enumeration is gated to admin or self.** The investigating token (Owner-level on the Foundation group) cannot read `last_used_at` or `last_used_ips` fields for any other user's Personal Access Tokens via the API. Verification of any specific PAT's usage history requires either the token owner pulling their own data from their user settings, or a request to GitLab security with proof of authorized investigation. This limitation was worked around for the specific exposed PAT via the out-of-band capture described above, but it is a significant general-purpose forensic capability gap that the Foundation should be aware of.
- **Push events on Premium tier do not include source IP addresses.** Push event analysis was therefore limited to author identity, timestamp, and commit hash continuity. A sophisticated attacker who routed PAT-authenticated requests through a VPN exit matching a team member's expected geographic pattern would not be detectable from the push event feed alone. Other layers of evidence (per-commit content review, pipeline forensics, and the PAT usage record) hold against this scenario.
- **The investigation cannot verify content of materials Chris may have read but not used to mutate state.** Read access via the exposed credentials could have included repository content, CI variable values via the API, project metadata, internal merge request discussions, deploy keys, and job log artifacts. None of these reads generate audit events. The investigation can verify that no mutating operations occurred under the affected identity from anomalous source addresses, but it cannot enumerate the full set of materials read.

2.6 Key Security Concerns

The following key security concerns were identified as the pre-investigation hypotheses to be tested. The investigation was structured to surface evidence for or against each hypothesis explicitly.

- **Code injection via stolen credentials:** the leaked Personal Access Token granted Maintainer-level write access to all seven Foundation repositories. The primary question was whether any party who scraped the token from a public IPFS bundle had used it to inject malicious code into the dApp source repository. Verification surfaces: per-commit content review, push event analysis, pipeline forensics, sibling project audits, IP geolocation under the affected identity.
- **Identity impersonation via unsigned commits:** the absence of commit signing across the entire repository history means commit author



metadata is not cryptographically verifiable. A stolen PAT could push commits attributed to any identity. Verification surfaces: identity audit, off-hours pattern analysis, IP-layer cross-reference of state-changing operations.

- **CI/CD pipeline as secret extraction vector:** with Developer-level access to the project (which can be obtained by various means), and CI/CD variables marked unprotected, an attacker could create a feature branch, modify the CI configuration to add a job that prints all environment variables, push the branch, trigger the pipeline, and read the secrets from the resulting job log. Verification surfaces: pipeline source/user/branch analysis, `.gitlab-ci.yml` modification history, suspect-account branch and pipeline check.
- **Force-push and history rewriting:** with Maintainer-level write access via the leaked credentials, history rewriting could have been used to remove the trace of an inserted commit. Verification surfaces: `git fsck` object integrity, push event commit hash continuity check against current git history, orphan commit detection.
- **Supply chain compromise via dependency injection:** the build configuration loads dependencies from `package.json` and `yarn.lock`. A compromise that added a malicious dependency, or modified an existing dependency's resolved URL to point at an attacker-controlled tarball, would persist across builds. Verification surfaces: `package.json` review, `yarn.lock` resolved-URL audit, install lifecycle script review.
- **Sentry-channel data exfiltration:** if error reporting is configured to capture local-variable values from stack traces and send them to a third-party error monitoring service, and if any code path ever logs an error containing sensitive material, that material would be transmitted to the configured Sentry organization. Verification surface: Sentry SDK initialization review, `beforeSend` hook presence check, integration list review.
- **Wallet engine private key leakage via inter-process communication:** the dApp uses Web Workers and BroadcastChannel for inter-context communication. If Keypair objects are serialized over a BroadcastChannel that other code can listen to, or if a debug log on a sensitive payload is captured by the error reporting integration, private key material could leak. Verification surface: full data flow trace of Keypair objects, BroadcastChannel listener inventory, debug log review.
- **Dormant attacker access:** a sophisticated attacker with temporary access via stolen credentials could plant a persistent backdoor: a new SSH key on a compromised account, a new long-lived Personal Access Token, an additional bot account with elevated permissions, a malicious webhook, or an extra Foundation member added to the group with quietly elevated role. Verification surface: SSH key audit, PAT enumeration

(within API limitations), member addition/removal audit, webhook inventory, deploy token review.

3 Report Summary

This engagement identified a total of **11 security findings** across the audited dApp repository, build pipeline, GitLab infrastructure, and supporting configuration. The findings are categorized by severity level as follows:

Severity Level	Count	Remaining	Finding IDs
Critical	1	0	PP-01
High	5	2	PP-02, PP-03, PP-04, PP-05, PP-10
Medium	3	0	PP-06, PP-07, PP-08
Low	1	0	PP-09
Informational	1	0 (partial)	PP-11

In addition to the findings, this report includes a dedicated **Forensic Verification** section documenting the methodology and outcome of the investigation into whether the credentials exposed by PP-01 were used to inject malicious code into the dApp source repository during the exposure window. The verification's conclusion is that no evidence of malicious code injection was found across any of the audited surfaces, with explicitly acknowledged limitations regarding read-only API operations and the pre-May-2025 audit log gap.

4 Audit Findings

4.1 [PP-01] Build pipeline serialized all CI/CD environment variables into deployed bundles

Risk Level

Severity: Critical, Likelihood: High, Status: Remediated

Discovery

PP-01 was externally disclosed to the Foundation by a third-party researcher prior to Arcadia's engagement. Arcadia's scope on this finding was verification of the remediation and investigation of whether the exposed credentials had been used during the exposure window. The remaining findings (PP-02 through PP-11) were identified by Arcadia during the broader review.

Code Location

- `dapp/webpack.config.ts` (the affected line, since removed)

Description

The dApp’s webpack configuration used the `DefinePlugin` in a pattern that serialized the entirety of `process.env` into the static JavaScript bundle at build time:

```
new webpack.DefinePlugin({  
  'process.env': JSON.stringify(process.env),  
  '__DAPP_CONFIG__': getDAppConfig(),  
}),
```

This is a well-known webpack misconfiguration. The intended use of `DefinePlugin` for environment variable injection is to enumerate explicit keys (e.g., `'process.env.NODE_ENV': JSON.stringify(process.env.NODE_ENV)`), exposing only the specific variables the runtime requires. The bulk pattern shown above instead serializes *every* environment variable present at build time, including, in this case, every CI/CD secret injected into the build environment by the GitLab CI runner.

The dApp build runs in a CI environment where the runner exports all configured CI/CD variables as environment variables before invoking the build. This included a Personal Access Token with Maintainer-level access to all seven repositories under the Foundation’s GitLab organization, third-party storage credentials (one IPFS pinning service, one S3-compatible service), error monitoring tokens (the Sentry organization auth token and several DSN keys), an RPC provider project ID and secret, and bundler service API keys for an ERC-4337 user operation submission service. All of these credentials were embedded in plaintext in the resulting JavaScript bundle.

The dApp’s deployment process pins each new build to IPFS and references it from a public IPFS gateway. Because IPFS-deployed bundles are content-addressed and immutable, every historical build that contained the leaked credentials remains accessible via its IPFS content hash even after the underlying configuration is corrected. The exposure persisted for approximately 16 months from November 2024 (when the leaking pattern was introduced during a routine package upgrade) until March 2026 (when the misconfiguration was identified and the line removed).

Impact

Any party who downloaded any historical IPFS bundle of the dApp during the 16-month window could extract the embedded credentials. The most consequential of the exposed credentials was the GitLab Personal Access Token, which carried the `api` scope and Maintainer-level access to all seven Foundation repositories. With this token, a holder could in principle: read the full source of every Foundation repository (including any not publicly hosted), modify branches that lack protected branch rules, modify CI/CD configuration, modify CI/CD variables, add new members to projects, create new bot tokens to maintain persistent access, modify branch protection settings, and modify project visibility. Whether any of these write capabilities was actually used is the subject of the Forensic Verification section of this report.



Recommendation

Replace the bulk `process.env` serialization with an explicit per-key allowlist:

```
new webpack.DefinePlugin({
  'process.env.NODE_ENV': JSON.stringify(process.env.NODE_ENV),
  'process.env.TARGET_ENV': JSON.stringify(process.env.TARGET_ENV),
  // ... only keys actually needed by the runtime
  '__DAPP_CONFIG__': getDAppConfig(),
}),
```

In addition: add a build-time guard that fails the build if `process.env` is referenced as a whole in `DefinePlugin` (a simple lint rule or pre-build check). Audit the historical IPFS bundles, either by unpinning them where the IPFS pinning service permits it, or by formally noting their continued accessibility in the public post-mortem and rotating any credential that has not yet been rotated. Consider a CODEOWNERS rule (see PP-05) that requires multi-Maintainer review for any change to `webpack.config.ts`.

Remediation

Remediated in #5ac1c129. The single offending line was removed from `webpack.config.ts`. Subsequent builds no longer embed the CI/CD environment in the deployed bundle. The Foundation rotated the affected credentials within hours of disclosure. Historical bundles on IPFS remain accessible via their content hashes; the exposed credentials embedded in those historical bundles remain exposed and must be considered permanently compromised regardless of rotation.

4.2 [PP-02] Default branch is not a protected branch

Risk Level

Severity: High, Likelihood: Medium, Status: Planned

Configuration Location

- GitLab project settings > Repository > Protected branches

Description

The dApp project's default branch (`main`) is not a protected branch in GitLab. The only branch with a protection rule is a leftover migration branch from May 2025 (`fix-package-names-20250522_223649`) which was the temporary default during the project migration into the current GitLab namespace and was never updated when the default was changed to `main`.

The practical consequences are: any user with Developer-or-higher access can push directly to `main` without going through a merge request, force-push to `main` without restriction, and bypass any review or approval gate. The repository's push history confirms this is not theoretical: 43 direct-to-`main` pushes



were observed during the audited window. While all 43 were performed by legitimate team members, the absence of branch protection means there is no architectural barrier preventing an unauthorized direct push if any contributor account or token were compromised.

Combined with PP-03 (no commit signing) and PP-04 (CI/CD variables unprotected), the unprotected default branch is the linchpin of the repository's overall write-side security posture: a stolen credential with sufficient scope can ship code to production via direct push, with no review and no signature verification, and the resulting build runs with full access to unprotected CI/CD secrets.

Recommendation

Configure protected branch rules on `main`:

- Allowed to push: Maintainers only
- Allowed to merge: Maintainers only (or, with PP-05's CODEOWNERS in place, require approval from a code owner)
- Allow force push: false
- Code owner approval required: true (paired with PP-05)

Apply equivalent rules to any other production-target branches (e.g., `canary`, `staging`, `production-*`) if they exist or are added later. Remove the stale `fix-package-names-20250522_223649` protected branch entry if the branch is no longer in use.

Remediation

Planned. Protected branch rules on the default branch are scheduled for a later hardening phase and are not yet in place.

4.3 [PP-03] No commit signing required or in use

Risk Level

Severity: High, Likelihood: Medium, Status: Partially Remediated

Configuration Location

- Repository-wide; GitLab group setting `web_based_commit_signing_enabled`

Description

Across the entire history of the dApp repository (3,988 commits), zero commits carry GPG or SSH cryptographic signatures. `git log --pretty=format:'%G?' | sort | uniq -c` returns 3988 N (no signature) and zero G (good signature). The signing baseline for the repository is 0%.



The implication is that commit author metadata cannot be verified. Git's commit `author` field is a free-form string set by `git config user.name` and `user.email` on the committing machine, with no cryptographic binding to the actual author. A stolen Personal Access Token can push commits whose author field is set to any identity (including a forged identity matching a real team member's name and email), and the commit is indistinguishable from a legitimate one in its metadata.

This is the reason the per-commit content review (described in the Methodology section) had to use content-based criteria (the four-gate test) rather than signature-based criteria. Signature-based detection of impersonation is not available for this repository.

Recommendation

Enable required signed commits in GitLab branch protection on `main` (and any other protected branches added per PP-02). Each Foundation developer should configure GPG or SSH commit signing on their local machines: the setup is approximately ten minutes per developer and is documented in GitLab's official documentation. Configure the pre-receive hook to reject unsigned commits to protected branches.

For the historical commits (which cannot be retroactively signed without rewriting history), the post-incident state is: signing is enforced from the cutover date forward, and the historical commits are documented as "unsigned, predating the signing requirement."

Remediation

Partially remediated. Commit signing is now enabled and enforced for new commits going forward. Historical commits cannot be retroactively signed without rewriting history and are documented as predating the signing requirement, so signature-based verification is available from the cutover date onward.

4.4 [PP-04] CI/CD variables marked unprotected

Risk Level

Severity: High, Likelihood: Medium, Status: Remediated

Configuration Location

- GitLab project settings > CI/CD > Variables

Description

All CI/CD variables on the dApp project are marked masked but not protected. This includes the GitLab access tokens used by the build pipeline, the error monitoring auth token, and the third-party storage credentials. The



GitLab variable model has two distinct flags: **masked** (the variable's value is redacted from job log output where possible) and **protected** (the variable is only available to pipelines running on protected branches). The project's variables are masked but not protected, meaning they are available to pipelines running on *any* branch, including feature branches created by Developer-level users.

The exfiltration path enabled by this configuration is: any user with Developer access creates a feature branch, modifies `.gitlab-ci.yml` to add a job whose script prints all environment variables (or a specific subset of them, or a single targeted variable), pushes the branch, the runner executes the job on the feature branch with the unprotected variables in scope, the secrets are present in the job log, and the user reads the secrets from the resulting job log. Note that variable masking is a best-effort log redaction, not a security boundary; it can be circumvented by transformations such as base64 encoding the variable before printing.

While the audit found no evidence that this exfiltration path was actually exercised during the window (see the CI/CD pipeline forensics in the Forensic Verification section), the configuration represents a structural weakness that compounds the impact of any future credential or access compromise.

Recommendation

Mark all CI/CD variables as protected. Combined with PP-02 (protecting the `main` branch), this ensures that the variables are only made available to pipelines running on the protected branch, not to feature branches that any contributor can create. For variables that legitimately need to be available to feature branches (typically build configuration like `NODE_ENV`), use unmasked, non-secret values; never store secrets in unprotected variables.

Remediation

Remediated on GitHub. Following the repository's migration from GitLab to GitHub, the CI/CD secrets were re-provisioned as protected GitHub Actions secrets rather than unprotected variables, and the previous unprotected GitLab variables are no longer in use.

4.5 [PP-05] No CODEOWNERS rules enforce review on security-sensitive files

Risk Level

Severity: High, Likelihood: Medium, Status: Acknowledged (Risk Accepted)

Configuration Location

- Repository (no `CODEOWNERS` file present)



Description

The dApp repository does not contain a CODEOWNERS file, and no review-approval enforcement is configured for changes to security-sensitive files. The original credential exposure documented in PP-01 was introduced during a routine “Update Deprecated Packages” commit and shipped without external review. It then persisted for 16 months. A CODEOWNERS rule requiring two-Maintainer approval for changes to the build configuration would have routed the offending change through review and would, with reasonable probability, have caught the misconfiguration at the diff stage rather than after a 16-month exposure.

CODEOWNERS rules are particularly valuable for files where the security implications of a change are non-obvious to a routine reviewer (build configuration, CI configuration, key handling, error reporting, network connectors), and where the cost of mandatory review is small relative to the impact of an undetected change.

Recommendation

Add a CODEOWNERS file at the repository root that requires Maintainer-level review on the following paths (at minimum):

- `dapp/webpack.config.ts` and any other build configuration files
- `.gitlab-ci.yml` and any included CI configuration files
- `dapp/scripts/` (deployment scripts)
- `dapp/package.json`, `package.json`, and `yarn.lock`
- `dapp/src/services/keys.ts` and any other key handling files
- `dapp/src/services/sentry.ts` (or wherever Sentry is initialized)
- `dapp/src/services/connectors/` (RPC and wallet connector configuration)
- `dapp/src/workers/` (worker code, particularly the wallet engine workers)
- `crypto/` (cryptographic primitives)

Pair the CODEOWNERS rule with a protected branch rule (PP-02) that requires code owner approval before merge to `main`. The combination ensures that no change to a security-sensitive file can ship to production without explicit Maintainer review.

Remediation

Acknowledged. The repository currently has a single active contributor, so a CODEOWNERS review-routing rule provides no separation-of-duties benefit at this time and is not being added. The Foundation should revisit this finding



and add CODEOWNERS rules for the security-sensitive paths listed above if and when the contributor set grows beyond one.

4.6 [PP-06] Sentry initialization lacks beforeSend hook

Risk Level

Severity: Medium, Likelihood: Low, Status: Acknowledged (Risk Accepted)

Code Location

- `dapp/src/index.tsx` (Sentry SDK initialization, approximately lines 18-32)

Description

The Sentry SDK is initialized without a `beforeSend` hook:

```
Sentry.init({
  dsn: dAppConfig.sentryDsn,
  integrations: [
    new BrowserTracing(),
    new CaptureConsole({ levels: ['error'] }),
  ],
  tracesSampleRate: 1,
  enabled: !isDevelopment,
  beforeBreadcrumb(breadcrumb) {
    if (breadcrumb.level === 'debug') return null;
    return breadcrumb;
  },
  // No beforeSend hook
});
```

The `CaptureConsole` integration with `levels: ['error']` forwards every `console.error()` call as an event to the configured Sentry organization. Without a `beforeSend` hook, the contents of the captured event (including local variable values present in the stack trace, the message string, and any object passed to `console.error`) are sent verbatim to Sentry's servers.

The risk is that any code path that ever logs an error containing sensitive material (a private key, a mnemonic seed, a ZK proof witness, a wallet address, transaction inputs) will result in that material being transmitted to and stored in the Sentry organization. The `dApp` handles users' private wallet keys, and the wallet engine performs operations whose error states might include sensitive context. While the audit found no specific code path that currently logs sensitive data via `console.error`, the absence of a `beforeSend` hook means there is no defensive layer if such a code path is added in the future, or if an existing code path begins to throw errors with sensitive context due to a future change.

A related observation: the dApp also contains a debug log statement in the wallet engine worker (PP-09) that serializes the entire incoming message payload. Although that log is gated to development mode and is at `debug` level (which the configured `CaptureConsole` integration does not currently capture), the configuration is one settings change away from accidentally bridging the two issues.

Recommendation

Add a `beforeSend` hook to the Sentry initialization that:

- Strips local variable values from stack frame data
- Redacts the message string and any `extra` or `contexts` fields if they contain patterns matching sensitive data (hex strings of typical private key length, mnemonic word sequences, base64 strings of typical key length)
- Strips request bodies from breadcrumbs and contexts
- Drops any event whose unredacted message or extra data contains references to wallet private keys, mnemonics, or proof witness fields

Sentry's documentation provides reference implementations of `beforeSend` hooks for sensitive-data scrubbing.

Remediation

Acknowledged (Risk Accepted). The Foundation has reviewed this finding and accepts the current residual risk. The audit found no existing code path that logs sensitive material through the console levels captured by Sentry, so the exposure is latent rather than active. Adding a `beforeSend` scrubbing hook remains the recommended defensive measure and can be introduced if a future change alters this risk profile.

4.7 [PP-07] Two-factor authentication not required for group members

Risk Level

Severity: Medium, Likelihood: Low, Status: Remediated

Configuration Location

- GitLab group settings > Permissions and group features > Two-factor authentication

Description

The Foundation's GitLab group does not require two-factor authentication for member accounts (`require_two_factor_authentication: false`). Combined with the use of long-lived Personal Access Tokens for git and API op-

erations, and the absence of commit signing (PP-03), this means that any compromise of any contributor's GitLab account credentials yields direct API and git access without a second authentication factor.

Recommendation

Enable required two-factor authentication for all members of the Foundation group, with a reasonable grace period during which existing members must enroll. Communicate the change in advance to avoid unintentional lockouts. After the grace period, GitLab will require all members to have a second factor configured before they can access group resources.

Remediation

Remediated. Two-factor authentication is now enabled for all members, so account access requires a second authentication factor in addition to the account password.

4.8 [PP-08] Project Access Token with overly broad scope

Risk Level

Severity: Medium, Likelihood: Low, Status: Remediated

Configuration Location

- GitLab project settings > Access tokens (one specific token associated with an IDE integration)

Description

A Project Access Token associated with an IDE integration on the dApp project carries the scopes `[api, write_repository]`. The `api` scope grants full read and write access to the GitLab API for the project, and `write_repository` grants the ability to push code and modify branches via git. This is broader than necessary for typical IDE-side integrations, which generally require only repository read access for browsing source files and code intelligence.

If the IDE integration's credential storage is compromised (for example, by a security incident on the IDE vendor's side, or by malware that scrapes credentials from the developer's local credential store), the over-granted token would yield full API write access and code-push capability for the dApp repository.

Recommendation

Audit the actual capabilities the IDE integration requires, and downgrade the token's scope to the minimum needed (typically `read_repository` or `read_api` for browse-only features). If the IDE integration genuinely requires `write_repository` for its functionality, accept the risk explicitly and document why. As a separate hygiene measure, audit all other Project Access To-



kens and Group Access Tokens (one additional dApp project token; no group tokens currently exist) for similar over-grants.

Remediation

Remediated. The over-scoped GitLab Project Access Token was retired during the migration from GitLab to GitHub and is no longer valid. Any equivalent integration on GitHub is provisioned with the minimum scope required for its function.

4.9 [PP-09] Development-mode logging serializes wallet keypair structures

Risk Level

Severity: Low, Likelihood: Low, Status: Acknowledged (Risk Accepted)

Code Location

- `dapp/src/workers/walletEngine/walletEngine.ts` (approximately lines 96-99)

Description

A debug log statement in the wallet engine worker serializes the entire incoming message payload via `JSON.stringify(event.data)` when `isDevelopment === true`:

```
if (isDevelopment)
  logger.debug(
    'Wallet engine message payload:
      ${JSON.stringify(event.data)}',
  );
```

The wallet engine receives messages over a `BroadcastChannel` from the main thread. One of the message types (the wallet initialization / scan message) has a payload that includes `Keypair` objects (specifically the root reading keypair, the root nullifier keypair, and the storage encryption keypair), each of which contains a `privateKey` field. The debug log therefore serializes private key material to the developer console when the `isDevelopment` flag is true.

The exposure is currently limited by two layers:

- The log fires only when `isDevelopment` is `true`, which is false in production builds.
- The configured Sentry `CaptureConsole` integration captures only `error`-level events, not `debug`, so the debug log output is not currently forwarded to the Sentry organization.



The risk is that a future change could break either layer (a developer accidentally building with development logging enabled in a production-like configuration, or the Sentry configuration being updated to include `debug` events) and the path would become an active sensitive data leak. The combination with PP-06 (no `beforeSend` hook) is the bridging concern: if both layers fail at the same time, private key material would be sent to and stored in the Sentry organization.

Recommendation

Replace the bulk `JSON.stringify(event.data)` with explicit logging of only the non-sensitive fields of the payload (e.g., the message type, the tab identifier, the chain ID, the action name). The full payload object should not be serialized to a log, even at debug level, even in development builds, when it can contain Keypair objects. As a defensive design rule: any log statement that touches a payload containing key material should be replaced with a structured logger that filters sensitive fields by type before serialization.

Remediation

Acknowledged (Risk Accepted). The Foundation has reviewed this finding and accepts the current residual risk. The debug log fires only when the development flag is true (which is false in production builds), and the Sentry Capture-Console integration captures only error-level output, not debug-level, so both mitigating layers hold in the current configuration. Replacing the bulk payload serialization with explicit non-sensitive fields remains the recommended fix should either layer change in the future.

4.10 [PP-10] Sibling repository CI/CD variables marked unprotected

Risk Level

Severity: High, Likelihood: Low, Status: Remediated

Configuration Location

- One sibling Foundation repository (separate from the dApp), CI/CD variables

Description

A sibling repository under the Foundation organization carries five CI/CD variables, all masked but unprotected. The variables include a GitLab API token, a project access token, and a third-party RPC provider secret. They were created in May 2025 and have not been modified since.

The same architectural concern as PP-04 applies: because the variables are unprotected, they are available to pipelines running on any branch of that reposi-

tory, including feature branches. The exfiltration path (modifying `.gitlab-ci.yml` on a feature branch to print the variables to a job log) is the same. While there is no evidence in the audit that this path was exercised on the sibling repository during the audited window, the variables share an architectural weakness with the dApp's variables and warrant the same hardening.

The Forensic Verification section confirms that the sibling repository did not exhibit any anomalous activity from suspect identities or source addresses during the audited window. The finding here is purely a hygiene and rotation concern, not evidence of active exploitation.

Recommendation

Rotate the credentials currently stored in the sibling repository's CI/CD variables (GitLab API token, project access token, RPC provider secret). Re-add them as protected variables after rotation, so they are only available to pipelines running on the protected branch of that repository. Apply consistent CI/CD variable hygiene policy across all seven Foundation repositories: every secret-bearing variable should be both **masked** and **protected**.

Remediation

Remediated. The sibling repository was migrated to GitHub, where its secrets were re-created as protected values and the affected credentials (the GitLab API token, the project access token, and the RPC provider secret) were rotated. The previous unprotected GitLab variables and the pre-rotation credentials are no longer valid.

4.11 [PP-11] Bundler API endpoint hardcoded with leftover debug logging

Risk Level

Severity: Informational, Likelihood: Low, Status: Partially Remediated

Code Location

- `dapp/src/services/bundler/core.ts`
- `dapp/src/services/connectors.ts`
- `dapp/src/components/Rewards/RechargePool/RechargePool.tsx`

Description

A commit during the audit window introduced a hardcoded RPC provider URL with embedded API key into the bundler service code:

```
const provider = new JsonRpcProvider(  
  // 'https://polygon-mainnet.g.alchemy.com/v2/...',  
  'https://polygon-mainnet.infura.io/v3/<key>',
```



```
);
```

with a commented-out alternative provider above the active line. The same key was already present in another file in the codebase, so the change is technically a duplication of an existing credential rather than a new exposure. The same commit also left a `console.log` statement in the recharge pool component that prints two reward calculation fields to the developer console:

```
console.log({ minRewardableAmount, releasableAmount });
```

The hardcoded RPC API key is a public-by-design rate-limit identifier, not a cryptographic secret in the conventional sense; the disclosure context lists this category of credentials as “still live” (i.e., the team has chosen not to revoke them because they are intentionally embedded in the client). Hardcoding such identifiers in client-side JavaScript is standard Web3 dApp practice. The finding here is therefore not a security incident; it is a code quality and review hygiene observation:

- The commit message describes a CI/CD configuration change but the diff also includes UI changes, contract method additions, and the hardcoded provider: a “kitchen sink” commit whose message does not describe its actual contents
- The leftover `console.log` indicates the commit shipped with developer debugging artifacts still in place, suggesting incomplete code review at the time of merge
- The commented-out alternative provider URL suggests the developer was experimenting with provider choices and did not clean up after settling on one

Recommendation

Remove the leftover `console.log` statement from the recharge pool component. Decide whether the bundler service should use a configuration-driven RPC provider rather than a hardcoded one (configuration is the better default; hardcoded is acceptable for public rate-limit identifiers but obscures changes during review). Remove the commented-out alternative provider URL. Adopt a code review convention that flags “kitchen sink” commits whose message does not describe most of the diff, and require such commits to either be split into smaller commits with accurate messages or to have their messages updated to enumerate every meaningful change before merge.

Remediation

Partially remediated in #a83e3351, which adds a bundler proxy server and removes embedded API keys from the dApp. The bundler proxy migration addresses the structural concern (hardcoded provider URL) by routing the request through a configurable proxy. The leftover `console.log` and the

commented-out provider URL were not addressed by the remediation commit and remain pending cleanup.

5 Forensic Verification

This section documents the methodology and outcome of the forensic verification work performed to determine whether the credentials exposed by PP-01 were used to inject malicious code into the dApp source repository or to extract additional secrets via paths beyond the original IPFS bundle exposure. The verification is the most consequential part of this engagement and is presented in detail commensurate with its importance.

5.1 Verification approach

The verification was structured around eight explicit hypotheses, each one stating a specific way in which the exposed credentials could have been misused, paired with the verification surfaces that would surface evidence for or against the hypothesis. The hypotheses and the surfaces were defined before the data was analyzed, so that the analysis could not be unconsciously shaped by what was already visible.

5.2 Hypotheses tested

- **H1: Code injection via stolen credentials.** A party who scraped the exposed Personal Access Token from a public IPFS bundle used it to push malicious code to the dApp source repository under any author identity. Verification surfaces: per-commit content review of every commit on the default branch within the exposure window; push event analysis (authorship, timing, branch ref distribution); commit hash continuity check between the events feed and the local git history; force-push and orphan-commit detection; build pipeline content audit; private key handling and IPC audit; network and exfiltration sink audit.
- **H2: CI/CD pipeline exploitation as a secret extraction vector.** A party with sufficient project access (legitimately granted Developer or higher, or via the leaked credentials) modified the CI configuration on a feature branch to print environment variables to a job log, ran the pipeline, and read the secrets. Verification surfaces: full pipeline run history with triggering user, source, branch, and status; `.gitlab-ci.yml` modification history with per-commit diff review; suspect-account branch and pipeline activity check; CI variable change audit.
- **H3: Force-push and history rewriting.** A party with write access used the leaked credentials to rewrite git history, removing the trace of an inserted commit, or replacing a legitimate commit with a malicious one. Verification surfaces: `git fsck` object integrity verification; commit hash



continuity check between the push events feed and the local git history; per-branch reachability analysis; orphan commit detection.

- **H4: Identity impersonation via IP-layer evidence.** The leaked credentials were used to perform any state-changing audit-event-generating operation under the affected account's identity from a source IP address that does not match the account holder's expected location. Verification surfaces: full audit-event collection across all seven Foundation projects; per-author IP enumeration and geolocation; cross-account IP overlap detection; team baseline construction.
- **H5: Direct authentication evidence (out-of-band).** The leaked Personal Access Token's IP usage history shows authentication from any source IP not consistent with the legitimate token owner's normal use. Verification surface: out-of-band capture of the token's IP usage record from the GitLab user interface, preserved with chain of custody, with each IP independently geolocated.
- **H6: Sibling repository compromise.** The leaked credentials were used to access or modify any of the six other Foundation repositories beyond the dApp. Verification surfaces: full audit-event and push-event collection for each of the six sibling projects; per-project member enumeration; per-project CI variable enumeration; suspect-account activity check; IP cross-reference.
- **H7: Persistent backdoor installation.** A party with temporary access via stolen credentials installed a persistent backdoor: a new SSH key on a compromised account, a new long-lived Personal Access Token, an additional bot or user account with elevated permissions, a malicious webhook, an extra group or project member with quietly elevated role. Verification surfaces: project access token enumeration across all seven projects; group access token enumeration; member roster comparison; webhook inventory; deploy token review; new-token-creation audit events.
- **H8: Read-only secret extraction beyond the IPFS bundle.** The leaked credentials were used to read CI/CD variable values, deploy keys, or other secrets via the GitLab API beyond what was already extractable from the public IPFS bundles. Verification surface: not directly verifiable from audit logs (read operations do not generate audit events on the GitLab tier in use); inferred from the absence of write activity under the affected identity (i.e., the credentials were used for read access only) and bounded by the set of materials accessible to the credentials' scope.

5.3 Evidence collected

The verification work generated the following raw evidence, all preserved in a forensic evidence directory with SHA-256 hashing for chain of custody:

- Forty group-level audit events covering the period the current group has existed
- 186 project-level audit events across all seven Foundation projects
- 523 push events for the dApp project across the exposure window
- 511 pipeline runs for the dApp project across the exposure window
- Job-level details for any pipeline matching suspect criteria
- Push events, member rosters, and CI variable enumerations for the six sibling projects
- Project Access Token and Group Access Token enumerations for all seven projects and the parent group
- Geolocation records (from two independent providers) for every unique source IP that appeared in any audit event
- Out-of-band IP usage record for the specific exposed Personal Access Token, captured from the GitLab user interface and preserved with chain of custody

5.4 Findings per hypothesis

H1: Code injection via stolen credentials (no evidence found)

The per-commit content review of every commit on the default branch within the exposure window (270 commits) identified no commits with malicious content. Several initial escalations from automated review surfaces were independently verified by Arcadia personnel and determined to be false positives: the original credential exposure misconfiguration (which fits the textbook profile of an honest mistake during a routine package upgrade); a legitimate Sentry organization rename and US-to-EU region migration consistent with GDPR data residency requirements (confirmed by the configured Sentry organization remaining in active use after the migration); and a hardcoded RPC provider configuration that uses a key already present elsewhere in the codebase and follows standard Web3 client-side practice. The build pipeline audit found no install lifecycle hooks beyond standard `husky install`, no suspicious dependencies, no install-time arbitrary code execution paths, and no `yarn.lock` resolved URLs pointing to non-standard registries. The network and exfiltration audit inventoried every network IO call site in the dApp's source tree and verified each against an allowlist of legitimate destinations; no `eval`, `Function`, dynamic `import`, or other suspicious primitive was found.

The push event analysis (described in detail under H3) confirms that the in-window git history is internally consistent and that no suspect identity authored any push.

H1 verdict: no evidence of code injection found, with high confidence within the documented coverage.

H2: CI/CD pipeline exploitation (no evidence found)

The full pipeline history for the dApp project across the exposure window contains 511 pipeline runs. The triggering source distribution is 464 `merge_request_event` and 47 `push`; *zero* pipelines were triggered via `api`, `trigger`, `schedule`, `web`, or `external` sources, the source types that would allow a pipeline to run without going through normal push or merge request flow. The set of distinct triggering users across all 511 pipelines is exactly four: three known Foundation team members and one earlier UI contributor whose access has long since been removed. None of the suspect identities triggered any pipeline; bot accounts (the IDE integration token and the post-disclosure remediation token) triggered zero pipelines.

The `.gitlab-ci.yml` modification history during the exposure window contains exactly four commits, all by known team members. The diffs of all four commits were inspected and contain no patterns matching environment variable printing, base64 encoding of secrets, `curl` with secrets in URLs, `cat .env`, or any other exfiltration pattern. The only variable-adjacent lines in the CI configuration are existence-check gates of the form `if [ -n "$VAR" ]; then echo "...is set"` which do not interpolate the variable's value.

The 38 CI variable change audit events from the dApp project audit log were each individually reviewed: 21 by the dApp's primary engineer from his expected source IPs (Brazilian residential IPv6) and 17 by another team member from his expected source IPs (Egyptian residential IPv4). No CI variable changes were authored by any suspect identity. The post-disclosure burst of CI variable changes on March 31 corresponds to the legitimate rotation activity performed by the team in response to the disclosure.

H2 verdict: no evidence of CI/CD exploitation found, with high confidence within the documented coverage.

H3: Force-push and history rewriting (no evidence found)

`git fsck -full -strict` on the local clone returned a clean result with no integrity errors, confirming that no historical commit had been tampered with at the git object level since the clone. The commit hash continuity check was performed by extracting every `commit_to` hash from the 523 push events feed and verifying its presence in the current local git history. All 43 direct-to-main pushes have their `commit_to` hashes present in the current main branch history, forming a fully-chained history with no rewritten commits. Of the 479 unique non-zero `commit_to` hashes across all push events, 384 are missing from the current local git, but all 384 missing hashes are on feature branches (e.g., short-lived bug-fix branches and personal feature branches), garbage-collected



after merge or branch deletion. None of the missing hashes is on `main`. This is normal git lifecycle behavior, not force-push artifacts.

The orphan commit detection check is the inverse: of the 333 commits in the local git history within the window, all 333 are reachable from some logged `commit_to`. There are zero orphan commits in the local git history. This rules out the scenario in which code was pushed via a path not captured in the events feed.

H3 verdict: no evidence of force-push or history rewriting found, with high confidence within the documented coverage.

H4: Identity impersonation via IP-layer evidence (no evidence found)

Every unique source IP that appeared in any audit event across all seven Foundation projects (77 IPs total after deduplication) was independently geolocated via two providers and classified against the team baseline. The most consequential sub-finding is for the affected account whose credentials were exposed: every audit event under that account's identity, across all seven projects (46 events in total, all on the dApp project), was authored from a Brazilian residential IPv6 address in the `2804::/16` block, distributed across three Brazilian residential ISPs. There are zero outliers. There is no audit event under that account's identity, anywhere in the audit data, from any non-Brazilian IP address.

For every other team member, the IP pattern is similarly consistent with their expected geographic profile, with two exceptions worth noting (and which are not findings): one team member shows two events from a known commercial VPN provider (Datacamp/CDN77 ASN, a Mullvad/NordVPN exit), and two events from the country of Georgia, both consistent with travel; another team member shows multiple country IPs corresponding to actual travel (Spain, Czechia, Austria, Netherlands) plus two commercial VPN exits consistent with security-aware VPN use. None of these exceptions involve suspect-attributed activity.

A specific check for IPs shared between two team identities (the strongest signature of credential theft via the lazy version of the attack) found zero such overlaps. Every IP in the dataset is associated with exactly one team identity. Four IPs in the dataset resolve to AWS EC2 in Ireland and Virginia; these are GitLab.com's own server-side infrastructure dispatched on certain server-side actions (notably merge request merges and project access token creation via the web UI), not user endpoints, and are documented as such.

H4 verdict: no evidence of identity impersonation found at the audit-event IP layer. Caveat: this rules out the lazy version of the attack (an attacker using the leaked credentials directly from a non-team IP and performing any state-changing operation). It does not rule out a sophisticated attacker who routed their requests through a VPN exit matching the affected account's

expected geographic pattern; that scenario is addressed by other layers (the per-commit content review and the H5 finding below).

H5: Direct authentication evidence (POSITIVE FINDING)

The IP usage record for the exposed Personal Access Token was obtained out-of-band from the token owner via the GitLab user interface and preserved with chain of custody. The record covers the most recent six unique source IP addresses from which the token was used. The token's metadata shows: scopes [read_user, read_repository, read_virtual_registry, read_registry, read_api, api], expiration date in mid-2026, and a last-used date of March 31, 2026 (the day the disclosure was made to the Foundation).

Geolocation of the six IPs returned the following classification:

- **Five of the six IPs** resolve to Google Cloud Platform US-East-1 (Google LLC, AS396982, North Charleston, South Carolina), with hostname pattern *.bc.googleusercontent.com. These IPs are accounted for as the legitimate use of the token by self-hosted GitLab CI runners that the token owner had configured to use this token for git operations: when the runners pull or push, the runner's GCP IP is recorded as the token's last-used IP. The token owner confirmed this attribution.
- **One of the six IPs** resolves to a residential broadband connection on a US ISP not used by the token owner. This IP is *not* accounted for by any legitimate use of the token by its owner. It is geographically unrelated to the token owner's known location and is on a network that the token owner does not use. The IP corresponds to an external party already known to the Foundation in connection with the original disclosure that triggered this engagement.

The disclosing party's own written submission to the Foundation, which initiated this engagement, includes an acknowledgement that its author authenticated to the GitLab API using the exposed credential to verify the credential's access scope. This acknowledged authentication event is consistent with, and provides independent corroboration for, the unaccounted residential authentication recorded in the token's usage panel. The residential network from which the unaccounted authentication originated is on the same internet service provider and in the same metropolitan area as other GitLab activity previously observed from the same external party under their own platform identity. The identification of the unaccounted IP therefore rests on three mutually reinforcing strands: the disclosing party's own written acknowledgement of use, the server-side authentication record from a residential network outside the token owner's footprint, and the network-level correlation between that authentication and other activity by the same party previously observed on the platform.

H5 verdict: positive finding. The exposed Personal Access Token was used

to authenticate to the GitLab API from a source address that corresponds to a party already known to the Foundation, distinct from the token owner. The available evidence does not indicate that any state-changing operations were performed under the affected account's identity from this source address (see H4); the token usage was consistent with read-only access during the visible window. Important caveat: the GitLab UI exposes only the most recent six unique IPs, not the complete usage history. The Foundation may wish to obtain the full usage history through GitLab security support to extend the visibility of this finding to the full token lifetime.

H6: Sibling repository compromise (no evidence found)

The audit events, push events, and member rosters for all six sibling Foundation projects were collected and reviewed. Findings:

- Zero audit events authored by the affected account in any of the six projects. The account whose credentials were leaked has no audit footprint outside the dApp project; if the leaked credentials had been used to perform any state-changing operation against any of the sibling projects, it would have appeared in this data and it does not.
- Zero audit events from the suspect external party's known source address in any of the six projects.
- Zero audit events authored by any of the suspect identities in any of the six projects.
- 227 push events total across the six projects, all by known team accounts in expected branches and timezones.
- 31 unique audit-event source IPs across the six projects, all classifiable to known team members in their expected geographies. No IP shared between two team identities.
- One brief membership addition to a sibling project that was reverted approximately 90 seconds later, both actions performed by the same legitimate team member from a known source IP: a recognized legitimate cleanup sequence rather than an intrusion.

H6 verdict: no evidence of sibling repository compromise found, with high confidence within the documented coverage.

H7: Persistent backdoor installation (no evidence found)

Project Access Tokens were enumerated across all seven Foundation projects: only the dApp project has any Project Access Tokens (two), both fully accounted for (one IDE integration token created for legitimate purposes by the responsible Maintainer; one token created on the day of the disclosure as part of the remediation work). No Group Access Tokens exist on the parent group.

No bot or token creation events occurred on any project outside the dApp during the exposure window.

The member rosters of all seven projects were enumerated. New member additions during the exposure window were each cross-referenced against the legitimate adding Maintainer's stated context. All new additions are accounted for. No new SSH keys, GPG keys, deploy tokens, or webhooks were added during the window beyond known legitimate ones.

H7 verdict: no evidence of persistent backdoor installation found, with high confidence within the documented coverage.

H8: Read-only secret extraction beyond the IPFS bundle (not directly verifiable; inference acknowledged)

GitLab Premium tier does not generate audit events for read-only API operations. As a result, the verification cannot directly enumerate which API endpoints were accessed via the exposed credentials. The available indirect evidence is:

- The H5 finding establishes that the credentials were used to authenticate from a non-team source address.
- The H4 finding establishes that no state-changing operations were performed under the affected account's identity from any non-team source address. The credentials were therefore used for read access only, within the documented coverage.
- The credentials' scope (`read_user`, `read_repository`, `read_registry`, `read_api`, `api`) permits, in principle, reading repository content, project metadata, CI variable values via the API, project access tokens, deploy keys, member lists, merge request and issue contents, and pipeline job logs across all seven projects to which the credentials had access.
- The set of materials that could have been read is bounded by what was accessible to the credentials' scope; what was actually read cannot be enumerated from the available data without additional support from GitLab security.

H8 verdict: cannot be directly verified from the data available within the engagement scope. The inference from H4 + H5 is that the credentials were used for read access during the window; the specific materials read cannot be enumerated. The Foundation may wish to obtain server-side request logs from GitLab security to bound this gap further.

5.5 Acknowledged limitations of the verification

The verification's conclusions must be interpreted against the following explicit limitations, each of which has been documented in the Methodology section



above:

- The first six months of the exposure window (approximately November 2024 through May 2025) are not in the current group's audit log because the dApp project was migrated into the current GitLab namespace in May 2025. Audit data for this period lives in whichever namespace the project was previously hosted in. Recovering this data is an action item for the Foundation that lies outside the scope of this engagement.
- Read-only API operations do not generate audit events on the GitLab Premium tier in use. The verification cannot directly enumerate the specific materials accessed via the exposed credentials beyond what was inferable from the credentials' scope and the absence of write activity.
- Personal Access Token enumeration via the API is gated to admin or self. The Foundation cannot independently audit member PATs without either direct user-level cooperation (each member pulling their own data from their user settings) or a request to GitLab security with proof of authorized investigation. This limitation was worked around for the specific exposed PAT via out-of-band capture.
- Push events on Premium tier do not include source IP addresses. A sophisticated attacker who routed PAT-authenticated push requests through a VPN exit matching a team member's expected geographic pattern would not be detectable from the push event feed alone. This scenario is addressed by other verification layers (per-commit content review and force-push detection), but the Foundation should be aware of the gap.
- The local repository clone reflects the state of the repository at the time of cloning. Any modifications made to the repository between the clone and the time this report is read would not be reflected in the verification.

5.6 Conclusion of the verification

Across the eight hypotheses tested, seven returned negative findings (no evidence) within the documented coverage and one returned a positive finding bounded by the limitations above. The cumulative read is:

- **No evidence of malicious code injection** into the dApp source repository during the exposure window, across any of the verification surfaces examined.
- **Direct evidence that the exposed credentials were used** to authenticate to the GitLab API from a source address corresponding to a party already known to the Foundation. The use was consistent with read access; no state-changing operations were observed under the affected identity from anomalous source addresses.
- **No evidence of compromise** of the six sibling Foundation repositories beyond the dApp.



- **No evidence of persistent backdoor installation** (no unauthorized SSH keys, no unauthorized PATs, no rogue bot accounts, no malicious webhooks, no settings modifications under any team identity).
- **Acknowledged gaps:** the pre-May-2025 portion of the exposure window (approximately the first six months); read-only API operations not captured by audit events; the precise enumeration of materials read via the exposed credentials.

Within the coverage documented above and subject to the acknowledged gaps, the verification’s findings are most consistent with the credential exposure documented in PP-01, while a serious security incident, not having resulted in the worst-case scenarios that its scope made possible: across the forensic surfaces reviewed, the observable evidence indicates use of the exposed credentials for read access by an external party, and no malicious code injection into the dApp source was found. This should be read as “no evidence found within the documented coverage” rather than “no incident occurred”; where the acknowledged gaps obscure visibility (most notably the pre-May-2025 portion of the exposure window and read-only API operations not captured in audit events), the data does not support a stronger claim, and the recommendations in section 6 are intended to narrow those gaps through cooperation with GitLab security support. The remediation already performed by the Foundation (the immediate webpack fix in commit `#5ac1c129` and the subsequent rotation of affected credentials) addresses the underlying vulnerability. The remaining hardening work in this report (PP-02 through PP-11) addresses the latent issues that compounded the impact of the original misconfiguration and that should be resolved to prevent recurrence.

6 Conclusion

This security review of the Panther Protocol dApp and the supporting GitLab infrastructure was triggered by an external disclosure of a credential exposure in deployed build artifacts. The underlying vulnerability (a webpack build pipeline misconfiguration that serialized all CI/CD environment variables into the deployed JavaScript bundle) was a textbook example of a well-known footgun that nonetheless persisted for approximately 16 months because of a combination of insufficient code review (no CODEOWNERS), insufficient branch protection (no protected default branch), and the absence of a build-time guard against the offending pattern. The Foundation’s immediate response (root cause fix within hours of disclosure, credential rotation, member access cleanup) was timely and well-executed.

The forensic verification work performed during this engagement found no evidence of malicious code injection during the exposure window. The exposed credentials were used to authenticate to the GitLab API from a source address corresponding to an external party already known to the Foundation, but the use was consistent with read access only; no state-changing operations under

the affected identity from anomalous source addresses were observed across any of the seven Foundation repositories during the visible portion of the exposure window. The acknowledged gaps in the verification (the pre-May-2025 period; read-only API operations not captured in audit events) are documented and the Foundation has been advised on how to close them via cooperation with GitLab security support.

The eleven findings documented in this report fall into three categories: the original credential exposure (PP-01, fully remediated); a set of repository hardening gaps that compounded the impact of the original exposure and that should be closed to prevent recurrence (PP-02, PP-03, PP-04, PP-05, PP-07, PP-10); and a set of code-level findings ranging from medium to informational that were surfaced during the broader review (PP-06, PP-08, PP-09, PP-11). With the recommendations in this report implemented, the dApp's security posture would be substantially stronger than it was at the time of the original incident, and the class of vulnerability that PP-01 represents would be prevented from recurring through both architectural and process controls.

7 Contact Information

For further inquiries or to schedule a follow-up audit, please contact:

Arcadia

Email: info@arcadia.agency

Website: <https://arcadia.agency/>

Telegram: <https://t.me/thearcadiagroup>

8 Disclaimer

This report has been prepared by Arcadia for the Panther Protocol Foundation. While reasonable professional care has been exercised in its preparation, Arcadia and its authors assume no liability for errors, omissions, or damages resulting from its use.

The work documented is a point-in-time review of the Panther Protocol dApp source repository and the supporting GitLab infrastructure as they existed during the engagement window (April 2026). Software systems and infrastructure configurations evolve; any change made after the engagement date is outside the scope of this report, and its conclusions should not be relied upon as an assurance of the current or future security of systems that have since been modified.

The findings and recommendations in this report do not constitute a guarantee of security against attack. Sound security posture is established through layered defensive measures, independent peer review, and operational discipline over time; no single review can establish it in isolation.

The forensic verification component of this report is bounded by the coverage



and limitations explicitly documented in the Methodology and Forensic Verification sections. Within those bounds, “no evidence found” is the strongest claim the data supports and is not equivalent to “no incident occurred”; readers should interpret the verification’s conclusions accordingly.

This report is technical in nature and does not constitute legal advice. Where it describes the conduct or access of parties other than the engaging client, it does so on the basis of the technical and server-side evidence reviewed. Any decisions regarding legal action, disclosure, regulatory notification, or similar matters should be made in consultation with qualified legal counsel.